Assignment 2:

- This assignment exposes students to using SQL techniques to handle and prevent data integrity issues, design FK constraints with RI actions, and use common table expression techniques when developing queries.

- Due date is published on Nexus for this Assignment.

Checklist: What to hand in?

Part A:

- ☐ A single database script (.sql file) for part A

Part B:

- ☐ A PDF of your completed data model showing RI action designs for B1
- ☐ A single executable database script for question B2

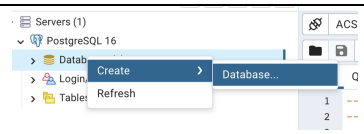
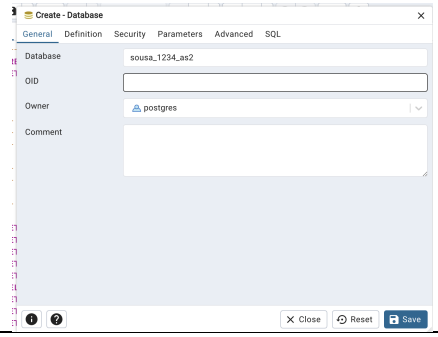
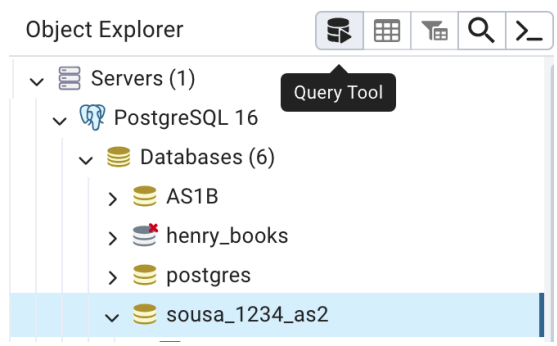
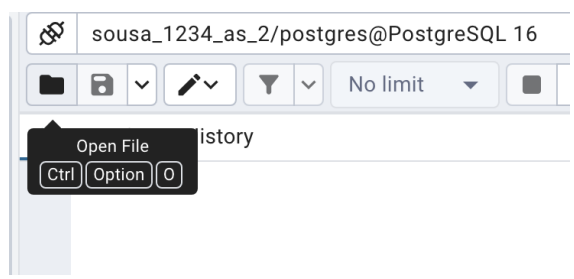
Part C:

- ☐ A single executable database script containing your queries.

IMPORTANT

- Each script file must have your name and student number (as comments) at the start of the file.
- You must use comments in your SQL scripts to separate and identify answers to each question
- Templates for each part of the assignment are available for download. You may use these to ensure your scripts are well-organized.
- A deduction of -5 for a script that fails to execute without errors may be applied. Ensure you test your work.
- Scripts without your name and student number may also receive a deduction.

Instructions: Database Setup for Part A and Part B

Instruction	Screenshot
Using pgAdmin, create a database within your Postgres environment to house assignment 1 Name the database with your name_st#_as2 Example: sousa_1234_as2 Refresh the object explorer to see the newly created database	 
Select the newly created database for and open a new query window/tool	 

In the query window, open the database setup script for assignment 2 part A.
 	
ACS-3902-Assignment-2-Part-A-B-Database-Setup.sql
 	
Change the first two lines of the script to match your name and student number and run the script to setup the database.
	`--- replace the schema name with your name/st#`
	`create schema if not exists sousa_1234_as2_a_b;`
	`set schema 'sousa_1234_as2_a_b';`

Part A: Constraints [35 marks]

Part A: Correcting and Preventing Bad Data

The database that has been setup for Part A simulates a database that has suffered from poor implementation and incomplete use of constraints to protect the integrity of the data. We are going to correct bad data then apply constraints to prevent future problems.

Incomplete Constraints

a. One of the primary keys were not declared.
b. No FK constraints were applied at all.

Data Quality Problems (due to bugs or poor design)

c. There are two customer records that, although they have a different customer ID, appear to be the same customer (a logical duplicate).
d. The cleaning invoice item table has two data quality issues:
    i. Duplicate invoice line item numbers within the same invoice.
    ii. Service Item Codes that don't exist in the Service Item table.

Tip: This section of the assignment modifies data. Students can always recreate the database and the original data to retry the script if regrettable errors.

A.1 Correcting Duplicate Data in the Cleaning Invoice Detail Table

A1.1 [5 marks] Create a Query to Show Duplicate Invoice Line Item Numbers

No primary key is declared on the Cleaning Invoice Detail table. Develop a query that shows which records would violate the primary key we intend to declare after correcting the data.

The query should show all the columns in the cleaning invoice item detail table but will only list the rows for invoices where the PK would be violated. Hint: use of a subquery may be required.

Explanation:

The intended primary key for the Cleaning Invoice Detail table is a composite key made up of the CleaningInvoiceNumber and the InvoiceItemLineNumber.

Normally invoices have sequential numbered line numbers that are unique (for that invoice). Some data exists where the line item number is repeated for the same invoice.

A.1.2 [5 marks] Correcting Bad Data in the Cleaning Invoice Item Table and Declaring the PK

Develop a series of update statements that will effectively correct the issue with the duplicate line item numbers.

Your script must meet the following requirements:

- The statements should be written within a SQL transaction (begin/end) to prevent partial correction in the event some statements succeed, and others fail.
- You may include several individual update statements to correct each row. Your design of the update statements can hard code criteria to isolate the invoice detail rows being corrected as required.
    - At the end of your script after closing the transaction, write a statement to add the PK using an alter table statement. The PK is a composite of the Cleaning Invoice Number and the Invoice Item Line Number.

A.2 Correcting Service Item Codes

A.2.1 [5 marks] Develop a Query to Show Data that Would Violate the Proposed FK Constraint

Develop a query that shows which records would violate referential integrity to the Service Item table and prevent us from implementing an FK constraint on service item id.

Your query should include the Customer ID, Business Name, Last Name, Invoice ID, Booking Date, Invoice Line Item Number, and the service item ID. Sort the query by customer ID and booking date.

A.2.2 [5 marks] Correcting Bad Data – Service Item Codes

Develop a series of update statements that will effectively correct the issue with the invalid service item codes on the cleaning service invoice detail table.

Agreed Upon Data Correction Approach
After meeting with our users, we have agreed to use the following corrective approaches:

- Codes that were accidentally entered with a leading zero by removing the leading zero (e.g., an incorrect code of '05' would be changed to a valid value of '5')
- Null codes are to be replaced with a code of 'UK' for unknown.
- Two additional service items are to be added (see below).

New Service Items to be Added

Service Item ID	Description	Unit Type	Standard Unit Price	Category
OT	Other Services	Hour	20	Other
UK	Unknown	Unknown	1	Other

Your script must meet perform the following:

- The statements to correct the data should be written within a SQL transaction (begin/end) to prevent partial correction in the event some statements succeed, and others fail.
- You may include individual update statement to correct each row (one at a time). Your design of the update statements can hard code criteria to isolate the invoice detail rows being repaired as required.
- After the end statement of the SQL transaction, write the SQL to add the FK constraint using an alter table statement. This will add the FK constraint between the cleaninginvoice item and the serviceitem tables which should now be successful since you have corrected the data issues.

A.3 Merging Duplicate Customer Records

A.3.1 [5 marks] Develop a Query to Show Potential Duplicate Customers
Think of and develop a query that will produce a list of candidate duplicate customers. We use the word "candidate" because, technically the customers have a different customer ID but based on their other information, they are likely the same customer. The data set provided has two customer records that can be interpreted as candidate duplicates.

The list should include the customer ID, type, business name, first/last name, address columns, and email and should be sorted by customer ID.

A.3.2 [5 marks] Merge Duplicate Customers
Develop a series of update statements that will effectively merge two of the duplicate customers into one record. You should keep the customer record that is judged to have more complete data.

Your script must meet the following requirements:

- the statement should be written within a SQL transaction (begin/end) to prevent partial correction in the event some statements succeed, and others fail.
- include a separate update statement for each of the related child tables that will associate the child records with the customer record you intend to keep.
- To ensure completeness you should write an update statement for all child tables of the customer table even if the affected customer(s) do not have related records in those tables.
- Your design of the update statements can hard code criteria as needed.
- The last step in the transaction will be a delete statement to remove the unwanted record from the customer table

A.3.3 [5 marks] Implement Constraints to Prevent Future Duplicate Customers
Examine the data and conceive of one or more constraints (as you see fit) for the customer table that meets the following requirements:

- Would have prevented the duplicate customer cleaned up in the questions above.
- Can be applied successfully to the existing data (after the customer cleanup procedure above has been completed). Hint: enforcing uniqueness on email address alone will not be sufficient.

Write the statement to add the constraint to the customer table using the ALTER TABLE command.

Part B: Establishing FK Constraints with Consideration to Update/Delete RI Actions [24 marks]

This part of the assignment uses the same database that was worked on for part A.

B1: [12 marks] Designing and Annotating Referential Integrity Actions in a Data Model

A starter model is available on Lucid Chart at this link. Make a copy of the provided information engineering model which has been partially completed. On your copy, revise your name and student number in the title. The model has all the PKs declared, has correct cardinality but has no FKs.

Revise the Model by:

Indicating all the FKs within the data model.

Annotate each relationship to indicate the appropriate RI actions
(see the appendix of this assignment or this document on Lucidchart for the notation to use) **

As you indicate the RI actions, ensure you choose reasonable settings given the nature of the data. In general, entities that represent financial transactions should never be deleted or orphaned if the parent entity is deleted or updated.

**** Lucid chart Drawing Tips to add RI Action Notation to Relationships**

Technique 1: To place an additional piece of text on the relationship connector, double click the relationship slightly away from the existing label.

Technique 2: Edit the existing label and place the RI actions on a separate line. To get a separate line in Lucidchart, use CTRL-ENTER while editing text on a relationship.

Once your model is complete, export a PDF version of it for submission.

B2: [12 marks] Creating FK Constraints with RI Actions

Completing this work requires that you have successfully completed part A1 and A2 (data corrections) as some of the FK constraints may fail if the data is incorrect.

Create a database script to add the constraints you designed in part B1. You should use ALTER TABLE statements to add the constraint and specify the appropriate RI actions in agreement with your design.

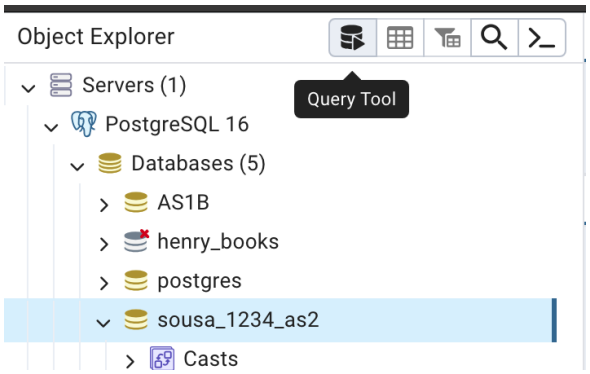
Special Exception:
In part A3.3 above you would have already created an FK constraint between the cleaninginvoiceitem and the serviceitem tables. You must drop that constraint before re-creating it with your referential integrity actions specified.

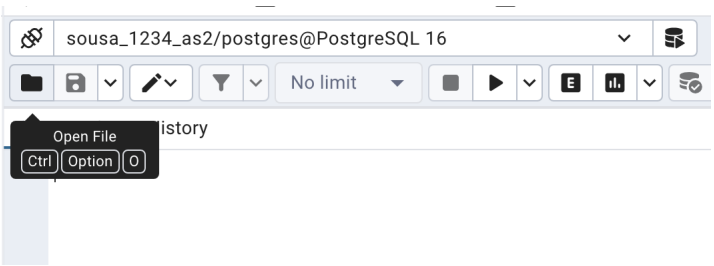
You must hand in your script for Part B2 as part of your assignment.

Part C: SQL Select Queries, Outer Joins, and CTEs [30 marks]

Setup the Database for Part C

The required SQL Script to create both the structure and data is available with this assignment and can be run in your Postgres environment. Setup instructions appear below.

Instruction	Screenshot
Click on your database for assignment 2 newly created database and open a new query tool/window.	

Click “Open File” and open the database setup file for Assignment 1 Part C provided with this assignment.
 	
Change the name and student number at the top of the script to ensure the database is created within an appropriate schema.
 	
Run the script and ensure no errors are encountered when you run the script.
	
	`-- Database Setup Script for ACS-3902 Assignment 2, Part C`
	`-- REPLACE YOUR STUDENT NUMBER AND NAME IN THE STATEMENTS BELOW`
	`create schema if not exists sousa_1234_as2_c;`
	`set schema 'sousa_1234_as2_c';`

The following questions are based on a hardware sales database. A diagram of the database is available at the end of this assignment.

Outer Join Queries [15 marks, 5 mark each]

Complete the following questions without the use of subqueries or CTEs.

1) Prepare a list of sales reps that do not have any customers currently assigned to them. Include the sales rep number, last name, and first name in the list.
2) Prepare a sales summary for each sales rep that provides the following summaries: the number of orders that their customers have placed, the total value of those orders (a sum of the quoted price * quantity sold on the order line). The final report should include rows for sales reps even if they don't have any customers or if customers that the sales rep represents do not have any orders.
3) Prepare a list of customers and a summary of their orders. The list will include the customer number, last name, the last name of their sales rep, and the average value of their orders (the order value is the (a sum of the quoted price * quantity sold on the order line). The final list must include customers even if they have no sales rep assigned and should be sorted descending by the average order value.

Common Table Expression Queries [10 marks, 5 marks each]

Complete the following queries using a common table expression technique with your query.

4) Produce summary of the orders for parts that have a unit price that is greater than or equal to the average unit price for all parts according to the unit price on the part table. The list should include the part number, part name, unit price, total quantity sold, and the total revenue generated (revenue is the sum of the quantity * quoted price). Sort the list descending by order quantity.

5) Produce a list of customers (customer number, first name, last name) along with a sum of all their order values (qty ordered * quoted price). Limit the list to only those customers who had purchased one or more products from the "SG" class. The summary of their orders, however, will include all products. Order the results by customer number. The correct results are shown below:

CUSTOMER_	Last	First	TotalValue
256	Samuels	Ann	162.45
311	Charles	Don	2709.9
315	Daniels	Tom	1119.96
522	Nelson	Mary	777.68

[5 marks] Insert Query based on a Select Query

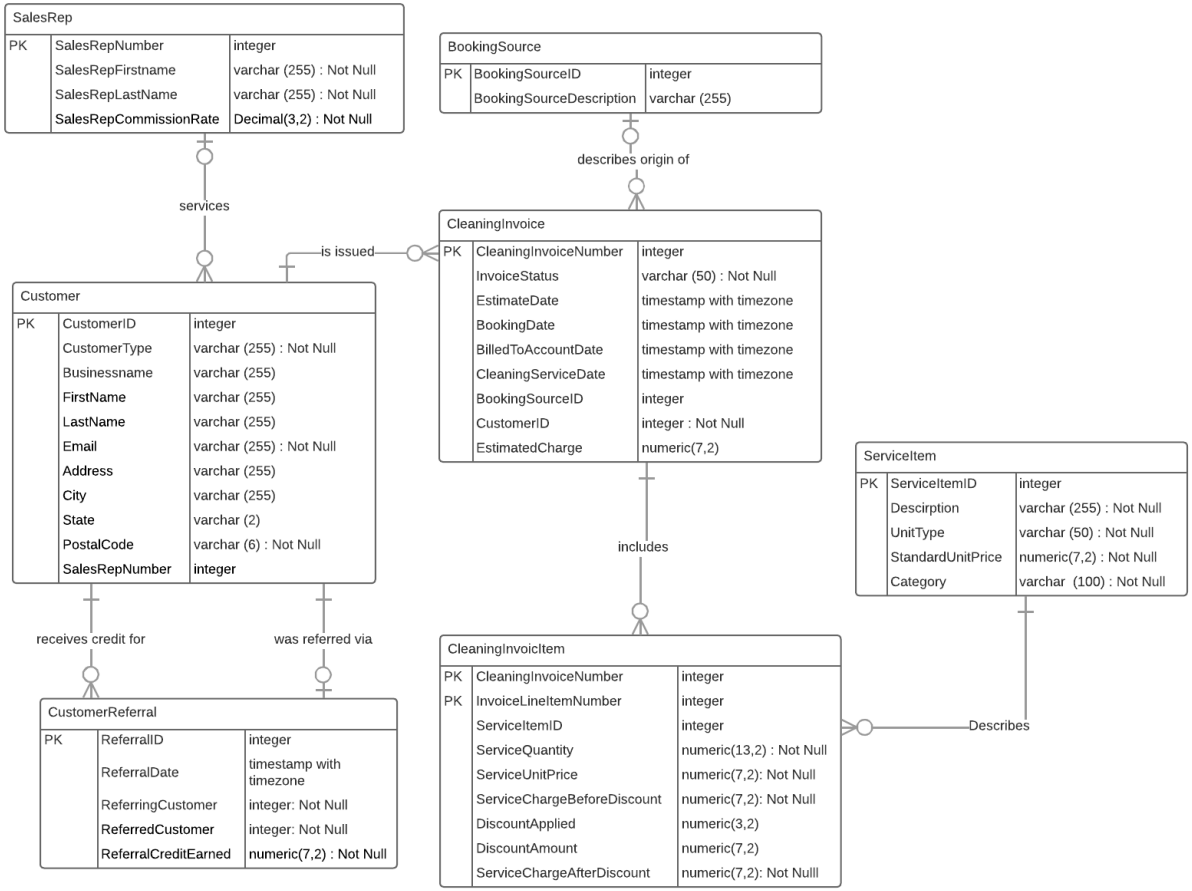
6) Prepare a database script that exists within a SQL transaction that will insert the sales commission results for a specific year and month (the query should include criteria that allows one to specify the year and month to prepare the commission payout results. The query should insert zero values for the total sales and the commission amount in cases where the sales rep had no sales attributed to them for that year/month.

   The Sales Commission Payout Table consists of the following attributes
   - Sales Rep Number
   - Commission Rate (taken from the sales rep table)
   - Sales Year (the year from the order date)
   - Sales Month Number (the month number from the order date)
   - The sum of the total sales (sum of qty ordered * quoted price from the order detail table)
   - The sum of the commission amount (sum of qty ordered * quoted price * commission rate)

Part B1: Starter Data Model in Lucidchart

Students may create a copy of this Lucid Chart Document to establish a starting point for Part B1 of this assignment.

3902: Assignment 2 Part B, Claudio Sousa, St#: 12345



Part B1: Notation for Showing RI Actions

The following notation is to be used for Part B of this Assignment. This notation guide is also available directly in Lucid Chart using this link

Indicating Referential Integrity (RI) Actions

As no definitive standards exist for indicating RI actions, the following convention used by some data modeling tools has been adapted for use in our course.

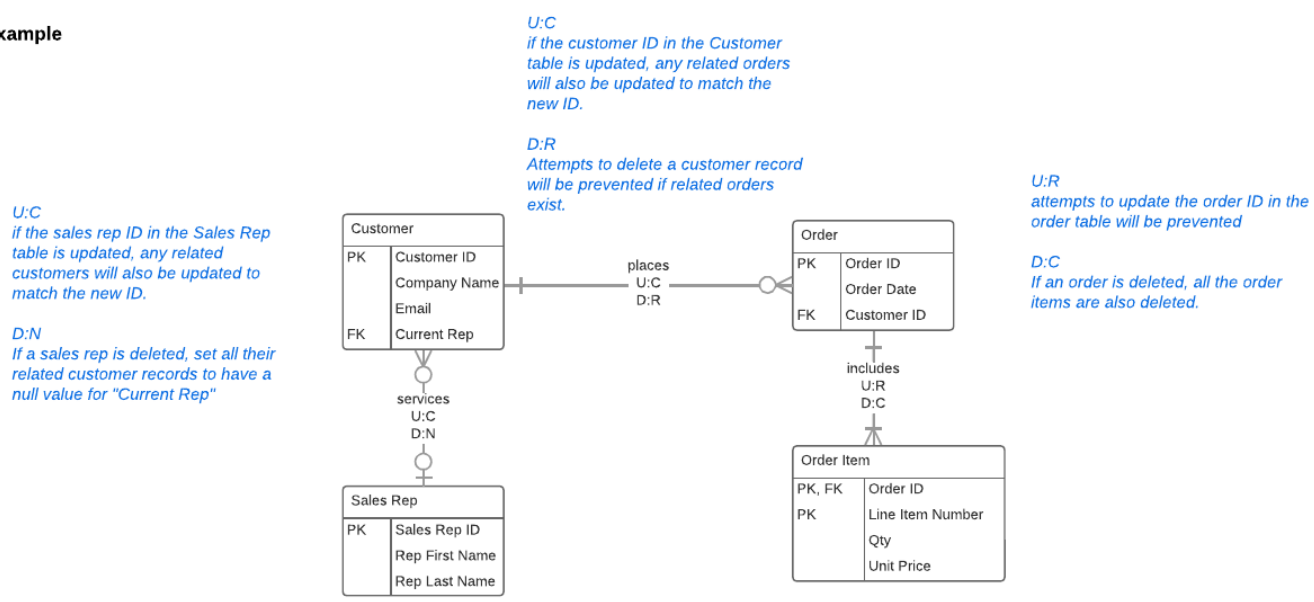
DML Event
U = Attempted **Update** of the parent PK.
D = Attempted **Delete** of the parent record.

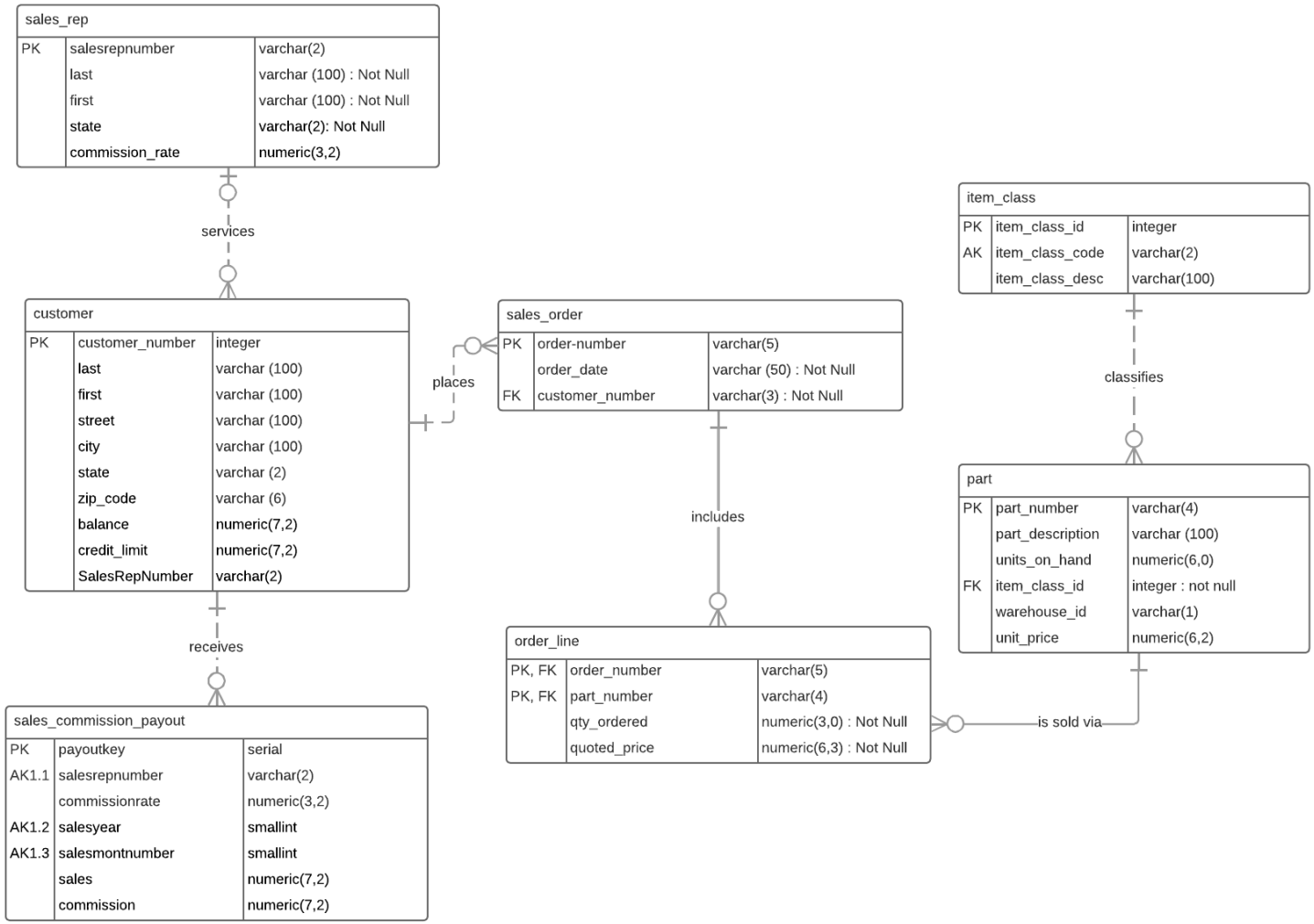
RI Action
R = **Restrict** the DML event from being processed.
C = Cascade the delete or update from the parent to the child.
N = Set the child FK values to **null** (only applicable to the delete DML event)

Valid Combinations

Annotation	Meaning	Explanation
U : R	on update restrict	prevents the parent PK from being updated when related child rows exist
U : C	on update cascade	allows the parent PK to be updated and updates any related child rows to match.
D : R	on delete restrict	prevents the parent row from being updated when related child rows exist
D : C	on delete cascade	allows the parent row to be deleted and also deletes any related child rows
D : N	on delete set null	allows the parent row to be deleted and sets the FK of any related child rows to null

Example



Part C: Data Model

3902: Assignment 2 Part C Data Model